# 22. Design Logging & Monitoring

**Key Terms:**

| Term | Full Name | What it does |
|---|---|---|
| **APM** | Application Performance Monitoring | Tracks app performance, errors, traces |
| **RED** | Rate, Errors, Duration | Key metrics methodology for services |
| **Trace** | Distributed trace | Request flow across multiple services |
| **Span** | Trace segment | Single operation within a trace |
| **P99** | 99th percentile | Latency that 99% of requests are faster than |
| **Sidecar** | Container pattern | Logging agent running alongside app container |
| **OpenTelemetry** | Observability framework | Vendor-neutral instrumentation standard |

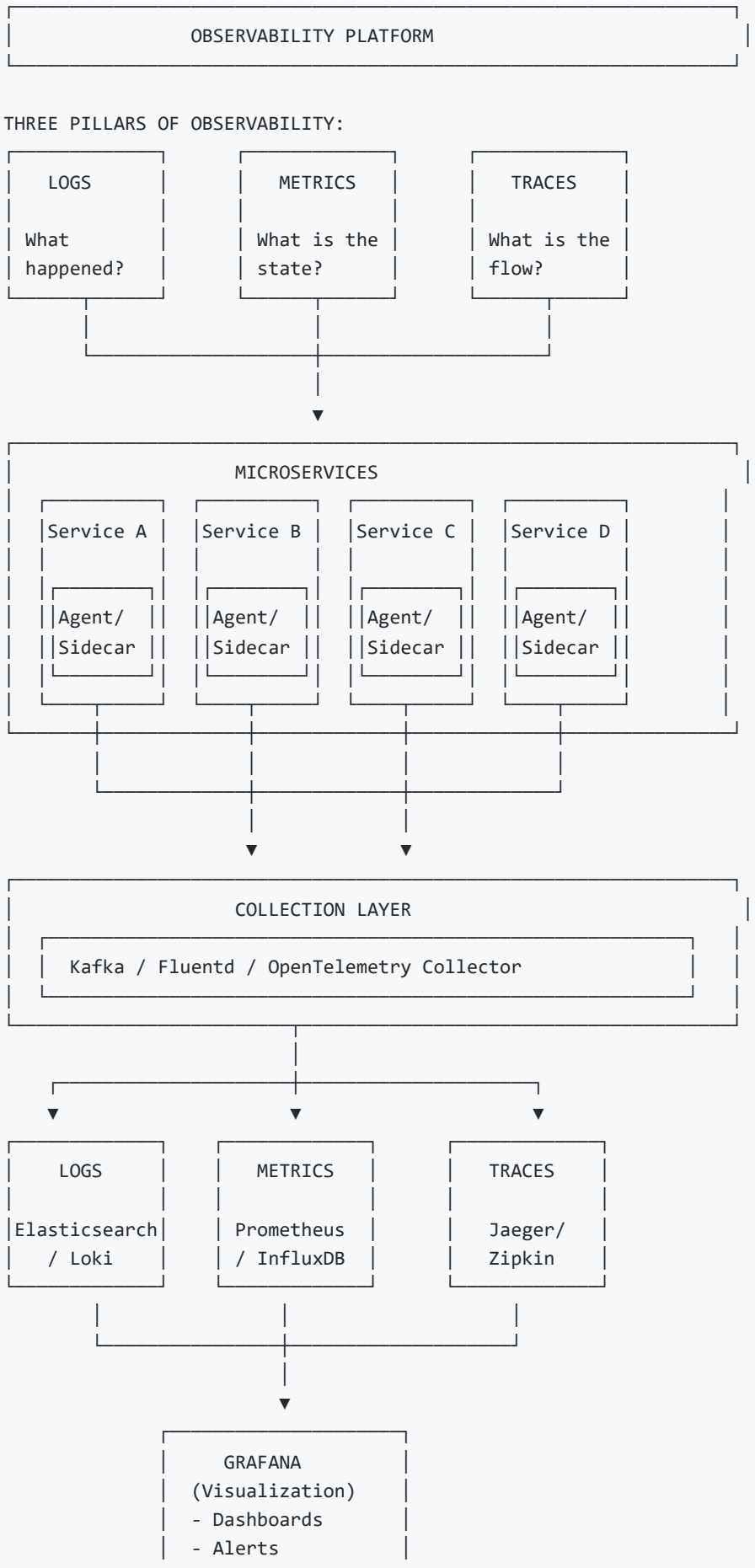
## Problem Statement

Design a logging and monitoring infrastructure for microservices that enables debugging, alerting, and performance analysis.

## Clarifying Questions to Ask

1. **Scale**: How many services? Log volume per day?
2. **Retention**: How long to keep logs? (30 days? 1 year?)
3. **Features**: Search, dashboards, alerting?
4. **Latency**: Real-time or near-real-time acceptable?
5. **Compliance**: Audit log requirements?
6. **Correlation**: Trace requests across services?
7. **Cost**: Budget for storage and compute?

## High-Level Design


OBSERVABILITY PLATFORM
THREE PILLARS OF OBSERVABILITY:
LOGS
What happened?
METRICS
What is the state?
TRACES
What is the flow?
MICROSERVICES
Service A
Service B
Service C
Service D
Agent/ Sidecar
COLLECTION LAYER
Kafka / Fluentd / OpenTelemetry Collector
LOGS
Elasticsearch / Loki
METRICS
Prometheus / InfluxDB
TRACES
Jaeger/ Zipkin
GRAFANA
(Visualization)
- Dashboards
- Alerts

```
                │  - Exploration      │
                └─────────────────────┘
```

## API Design

### Log Ingestion APIs

| Method | Endpoint | Description |
|---|---|---|
| `POST` | `/api/v1/logs` | Ingest log entries (batch) |
| `POST` | `/api/v1/logs/stream` | Stream logs via Server-Sent Events |

### Log Query APIs

| Method | Endpoint | Description |
|---|---|---|
| `POST` | `/api/v1/logs/search` | Search logs with filters |
| `GET` | `/api/v1/logs/tail?service={name}` | Live tail logs (SSE) |

### Metrics APIs

| Method | Endpoint | Description |
|---|---|---|
| `POST` | `/api/v1/metrics` | Push custom metrics |
| `GET` | `/api/v1/metrics/query?metric={name}&from={t1}&to={t2}` | Query time-series metrics |

### Alert APIs

| Method | Endpoint | Description |
|---|---|---|
| `POST` | `/api/v1/alerts/rules` | Create alert rule |
| `GET` | `/api/v1/alerts/rules` | List alert rules |
| `GET` | `/api/v1/alerts/active` | Get active alerts |
| `POST` | `/api/v1/alerts/{alertId}/acknowledge` | Acknowledge an alert |

### Tracing APIs

| Method | Endpoint | Description |
|---|---|---|
| `POST` | `/api/v1/traces` | Submit trace spans |
| `GET` | `/api/v1/traces/{traceId}` | Get full distributed trace |
| `GET` | `/api/v1/traces/search?service={name}&minDuration={ms}` | Search traces |

## Key Components

| Component | Purpose | Technology |
|---|---|---|
| Log Agent | Collect and ship logs | Fluentd, Vector |
| Log Store | Index and search | Elasticsearch, Loki |
| Metrics Collector | Scrape metrics | Prometheus |
| Metrics Store | Time-series storage | Prometheus, InfluxDB |
| Trace Collector | Distributed tracing | Jaeger, Zipkin |
| Visualization | Dashboards and alerts | Grafana |
| APM | Application performance | Application Insights |

## Structured Logging

```
LOG ENTRY (Structured JSON):
{
  "timestamp": "2024-01-15T14:30:00.123Z",
  "level": "ERROR",
  "service": "order-service",
  "version": "1.2.3",
  "trace_id": "abc123",
  "span_id": "def456",
  "user_id": "user_789",
  "message": "Failed to process order",
  "error": {
    "type": "PaymentDeclinedException",
    "message": "Card declined",
    "stack_trace": "..."
  },
  "context": {
    "order_id": "order_123",
    "amount": 99.99,
```

```
    "currency": "USD"
  },
  "host": "pod-xyz-123",
  "environment": "production"
}

LOG LEVELS:
  DEBUG → Development debugging (not in prod)
  INFO  → Normal operations
  WARN  → Potential issues
  ERROR → Failures requiring attention
  FATAL → System unable to continue
```

## Distributed Tracing

```
REQUEST FLOW ACROSS SERVICES:

┌──────────────────────────────────────────────────────────────────┐
│  User Request: POST /orders                                      │
│  Trace ID: abc123 (propagated through all services)              │
│                                                                  │
│  ┌──────────────────────────────────────────────────────────┐    │
│  │ API Gateway    [span: abc123-001] ──────────────────────┤    │
│  │   │                                                50ms  │    │
│  │   ▼                                                      │    │
│  │ Order Service  [span: abc123-002] ──────────────────┐    │    │
│  │   │                                            30ms  │    │    │
│  │   ├──▶ Inventory Service [span: abc123-003] ────┐   │    │    │
│  │   │                                        15ms  │   │    │    │
│  │   │                                              │   │    │    │
│  │   ├──▶ Payment Service   [span: abc123-004] ──┐  │   │    │    │
│  │   │                                    20ms  │  │   │    │    │
│  │   │                                          │  │   │    │    │
│  │   └──▶ Notification Svc  [span: abc123-005] ┐ │  │   │    │    │
│  │                                        5ms  │ │  │   │    │    │
│  └──────────────────────────────────────────────┴─┴──┴───┴────┘    │
│                                                                  │
│  Total: 50ms | Bottleneck: Payment Service (20ms)                │
└──────────────────────────────────────────────────────────────────┘

CONTEXT PROPAGATION:
  HTTP Header: traceparent: 00-abc123-def456-01
  Passes trace_id and span_id to downstream services
```

## Metrics Collection

```
KEY METRICS (RED Method):
┌──────────────────────────────────────────────────────────────────┐
│  Rate:   Requests per second                                     │
│  Errors: Error rate (%)                                          │
```

```
│  Duration: Latency percentiles (p50, p95, p99)                │
└──────────────────────────────────────────────────────────────┘

PROMETHEUS METRICS EXAMPLE:
  # Request count
  http_requests_total{service="order", method="POST", status="200"}

  # Latency histogram
  http_request_duration_seconds_bucket{le="0.1"} 1000
  http_request_duration_seconds_bucket{le="0.5"} 1500
  http_request_duration_seconds_bucket{le="1.0"} 1520

GOLDEN SIGNALS:
  - Latency: How long requests take
  - Traffic: How many requests
  - Errors: Rate of failed requests
  - Saturation: How full is the service (CPU, memory)
```

## Alerting Strategy

```
ALERT DEFINITIONS:
┌──────────────────────────────────────────────────────────────┐
│  CRITICAL (Page on-call):                                    │
│    - Error rate > 5% for 5 minutes                           │
│    - P99 latency > 2s for 5 minutes                          │
│    - Service completely down                                 │
│                                                              │
│  WARNING (Slack notification):                               │
│    - Error rate > 1% for 15 minutes                          │
│    - P95 latency > 1s for 10 minutes                         │
│    - Disk usage > 80%                                        │
│                                                              │
│  INFO (Dashboard only):                                      │
│    - Deployment completed                                    │
│    - Scaling event                                           │
└──────────────────────────────────────────────────────────────┘

ALERT FATIGUE PREVENTION:
  - Group related alerts
  - Suppress during maintenance windows
  - Escalation paths with timeouts
  - Runbook links in alert messages
```

## Scalability Considerations

- **Log volume**: Sample debug logs in production (1% sampling)
- **Retention**: Hot storage (30 days) → Cold storage (1 year)
- **Indexing**: Only index searchable fields, store rest as JSON
- **Cardinality**: Limit label cardinality in metrics

- **Correlation**: Use consistent trace_id across all telemetry

## Trade-offs

| Decision | Option A | Option B | Recommendation |
|---|---|---|---|
| Log storage | Elasticsearch | Loki | **Loki** - cheaper, label-based |
| Metrics | Prometheus | InfluxDB | **Prometheus** - ecosystem |
| Collection | Agent per node | Sidecar | **Sidecar** - Kubernetes native |
| Sampling | All traces | Sample 1% | **Adaptive** - all errors, sample success |

## Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

| Technology | Role in This Design | Reference |
|---|---|---|
| Kafka | Log collection and distribution pipeline | Tech Guide |
| Elasticsearch | Log indexing and full-text search | Tech Guide |

## How to Explain It Well

> **Opening**: "Observability requires logs, metrics, and traces working together. I'll design for debugging, alerting, and performance analysis."

> **Key talking points**:
>
> - "Structured JSON logs with trace_id for correlation"
> - "OpenTelemetry for vendor-neutral instrumentation"
> - "RED metrics: Rate, Errors, Duration for every service"
> - "Distributed tracing to identify bottlenecks across services"
> - "Alert on symptoms (error rate) not causes (CPU) to reduce noise"

---